

LABORATÓRIO 1 — ALGORITMOS DE ORDENAÇÃO

Implementação e Análise Comparativa de Algoritmos de Ordenação em Linguagem C

Objetivo

Este trabalho tem como objetivo desenvolver e consolidar os conhecimentos relacionados aos principais **algoritmos de ordenação**, por meio de sua implementação em linguagem C e da realização de experimentos que permitam analisar e comparar seus desempenhos e a relação entre a **complexidade teórica** e o **comportamento observado experimentalmente**, considerando diferentes volumes e organizações dos dados.

Descrição do Trabalho

Uma empresa necessita processar conjuntos de dados numéricos provenientes de diferentes fontes. Dependendo da origem, esses dados podem estar completamente desordenados, parcialmente ordenados, ordenados ou organizados em ordem inversa.

A equipe deverá desenvolver, em **linguagem C ou C++**, um sistema que permita gerar diferentes conjuntos de dados, aplicar algoritmos de ordenação e comparar seus desempenhos.

Deverão ser implementados os seguintes algoritmos:

- Selection Sort;
- Insertion Sort;
- Shell Sort;
- Quick Sort;
- Heap Sort.

Não será permitida a utilização de funções prontas de ordenação fornecidas por bibliotecas da linguagem C.

Funcionalidades do Sistema

O programa deverá apresentar um menu que permita ao usuário realizar, pelo menos, as seguintes operações:

1. Gerar um conjunto de dados;
2. Exibir os dados gerados;
3. Ordenar utilizando Selection Sort;
4. Ordenar utilizando Insertion Sort;
5. Ordenar utilizando Shell Sort;
6. Ordenar utilizando Quick Sort;
7. Ordenar utilizando Heap Sort;
8. Executar e comparar todos os algoritmos;
9. Encerrar o programa.

O sistema deverá permitir que o usuário determine o tamanho **N** do vetor a ser utilizado nos experimentos.

Geração dos Dados

O programa deverá ser capaz de gerar conjuntos de dados nas seguintes situações:

- dados aleatórios;
- dados previamente ordenados;
- dados em ordem inversa;
- dados parcialmente ordenados.

Para possibilitar uma análise adequada do comportamento dos algoritmos, deverão ser realizados experimentos com diferentes quantidades de elementos.

Sugere-se utilizar, no mínimo:

- 1.000 elementos;
- 10.000 elementos;
- 50.000 elementos;
- 100.000 elementos.

A equipe poderá utilizar outros tamanhos, desde que justifique sua escolha.

Instrumentação dos Algoritmos

Cada algoritmo deverá, além de realizar corretamente a ordenação, coletar informações sobre sua execução.

Deverão ser contabilizados:

- número de comparações realizadas;
- número de movimentações ou trocas de elementos;

- tempo de execução.

A equipe deverá definir uma estrutura (struct) apropriada para armazenar as estatísticas obtidas durante a execução de cada algoritmo.

Condições dos Experimentos

Para garantir uma comparação adequada, **todos os algoritmos deverão ser executados sobre conjuntos de dados idênticos.**

Assim, antes da execução de cada algoritmo, deverá ser realizada uma cópia do vetor original. Dessa forma, a execução de um algoritmo não poderá alterar os dados que serão utilizados pelos demais.

Os testes deverão considerar tanto diferentes tamanhos de vetores quanto diferentes organizações iniciais dos dados.

Apresentação dos Resultados

O programa deverá apresentar os resultados obtidos de maneira organizada, permitindo a comparação entre os algoritmos.

Uma possível apresentação é:

Algoritmo	Comparações	Movimentações	Tempo de execução
Selection Sort			
Insertion Sort			
Shell Sort			
Merge Sort			
Quick Sort			
Heap Sort			

Os resultados deverão ser coletados para diferentes tamanhos e diferentes organizações dos dados.

Análise dos Resultados

Além da implementação, a equipe deverá elaborar uma **análise dos experimentos realizados**, relacionando os resultados obtidos com os conceitos estudados sobre complexidade de algoritmos.

A análise deverá responder, entre outras, às seguintes questões:

1. Qual algoritmo apresentou melhor desempenho para conjuntos pequenos de dados?

2. Qual algoritmo apresentou melhor desempenho para grandes volumes de dados?
3. Quais algoritmos foram mais influenciados pela organização inicial dos elementos?
4. O Insertion Sort apresentou vantagens quando os dados estavam parcialmente ou totalmente ordenados? Explique.
5. O Selection Sort apresentou diferenças significativas entre dados ordenados, aleatórios e inversamente ordenados? Explique o resultado.
6. Os resultados experimentais estão de acordo com as complexidades teóricas estudadas?
7. Compare o comportamento do Quick Sort e Heap Sort. Quais vantagens e desvantagens foram observadas?
8. Existe uma relação direta entre o número de comparações realizadas e o tempo de execução? Utilize os resultados dos experimentos para justificar a resposta.

Entrega

A equipe deverá entregar:

- código-fonte completo em linguagem C;
- arquivos necessários para compilação e execução;
- relatório contendo a descrição da implementação;
- tabelas com os resultados dos experimentos;
- análise comparativa dos algoritmos;
- conclusões obtidas a partir dos experimentos.

O trabalho não deverá se limitar à implementação dos algoritmos. A **análise comparativa dos resultados obtidos** constitui parte fundamental da atividade e deverá demonstrar a compreensão das características, vantagens, limitações e complexidades dos diferentes métodos de ordenação.